# Design Justification Report

**Author:** Mike
**Course:** ECE 410/510
**Term:** Spring 2026
**Date:** May 20, 2026, Revision 1

## 1. Problem and Motivation

The primary objective of this project was to design, verify, and synthesize a custom hardware accelerator for a 3x3 convolution kernel, a fundamental operation in spatial image processing and computer vision. In a standard software implementation running on a general-purpose CPU, computing a 2D convolution over an image matrix requires deeply nested `for` -loops. Profiling this operation during the M1 baseline phase revealed severe inefficiencies in how the von Neumann architecture handles this specific workload. The software implementation was entirely memory-bound, achieving a throughput of merely ~0.05 GOPS.

The core issue stems from a lack of spatial and temporal locality exploitation in the software's memory hierarchy. A 3x3 convolution requires nine pixels for every single output calculation. As the kernel slides horizontally across an image, six of those nine pixels are identical to the pixels used in the immediate previous calculation. However, the CPU's standard L1/L2 cache mechanism is not inherently optimized for 2D sliding windows. As a result, the software was forced to repeatedly fetch overlapping boundary pixels from main memory, resulting in an operational intensity of just 0.25 Ops/Byte. This constant cache thrashing starved the ALU of data.

Custom hardware is necessary because a spatial convolution exhibits massive inherent parallelism and highly predictable data reuse patterns. By migrating this kernel to dedicated silicon, we can explicitly control the memory hierarchy and data flow. Instead of relying on general-purpose caches, we can build deterministic line buffers that keep required row data on-chip. This drastic reduction in external memory bandwidth requirements simultaneously increases computational throughput and vastly improves energy efficiency, justifying the engineering effort required for physical silicon implementation.

## 2. Roofline Analysis

The architectural design of the accelerator was directly driven by the roofline model analysis, which dictates the absolute physical limits of the target hardware platform. Our specific target environment has a peak computational ceiling of 1.8 GOPS and a memory bandwidth constraint of 0.4 GB/s. From these two parameters, we can calculate the ridge point—the exact operational intensity required to shift a workload from being memory-bound to compute-bound. Dividing the peak performance (1.8 GOPS) by the memory bandwidth (0.4 GB/s) establishes a ridge point of 4.5 Ops/Byte.

The M1 software baseline operated at an abysmal 0.25 Ops/Byte. Plotting this on the roofline graph places the software deep within the memory-bound region, far below the ridge point. It was physically impossible for the software to reach the computational ceiling because the memory interface was fully saturated long before the arithmetic logic units could be fully utilized.

To overcome this, the architecture was designed explicitly to shift the bottleneck. The goal was to push the arithmetic intensity past the 4.5 Ops/Byte ridge point. By implementing internal line buffers, the accelerator caches two full rows of image data on-chip. Once the initial latency period is overcome and the pipeline is primed, shifting the 3x3 window to the right by one pixel requires pulling only a single new byte (the leading edge pixel) from external memory. Using that single byte, combined with the

cached data, the compute core performs 9 Multiply-Accumulate (MAC) operations, which equates to 18 total arithmetic operations. This architectural decision successfully raised the arithmetic intensity from 0.25 to exactly 18.0 Ops/Byte. This massive leap pushes the design far past the 4.5 Ops/Byte ridge point, firmly placing it in the compute-bound ceiling and allowing the hardware to achieve the maximum 1.8 GOPS. *(See Figure 2: Roofline Plot)*.

## 3. Precision and Data Format

The accelerator utilizes 8-bit signed integer arithmetic for both the streaming input pixel data and the statically loaded filter weights. Inside the compute engine, the intermediate products are expanded, producing a 32-bit signed integer accumulation result before truncating or scaling back to the required output width.

This specific data format was chosen as the optimal balance between application accuracy and silicon area efficiency. As documented in the M2 precision analysis, standard software implementations default to IEEE 754 32-bit floating-point math. However, the physical area and dynamic power cost of a floating-point multiplier are orders of magnitude higher than an integer multiplier. Synthesizing nine parallel floating-point MAC units for the compute core would have violated the area constraints of the Sky130 node and severely impacted the critical path timing, likely forcing a drop in the target clock frequency.

Because 8-bit precision is the standard quantization level for grayscale image pixels, we performed an error analysis to verify if an 8-bit fixed-point weight quantization would degrade the image processing output. The verification proved that 8-bit quantized weights, when properly scaled, introduce less than a 2% degradation in the Structural Similarity Index Measure (SSIM) when compared to the floating-point baseline. For the target application domain of edge detection and spatial blurring, a 2% SSIM drop is visually imperceptible and represents an acceptable tolerance in exchange for massive savings in power, area, and timing slack.

## 4. Dataflow and Architecture

The core architecture implements a highly optimized sliding-window dataflow, which functions as a hybrid input-stationary and weight-stationary model. The 3x3 convolution weights are statically loaded into local configuration registers (weight-stationary), meaning they do not consume any active memory bandwidth during the continuous pixel stream. The input pixels are streamed into a custom memory hierarchy designed specifically for 2D spatial locality.

The memory hierarchy consists of two FIFOs acting as line buffers, augmented by a 3x3 grid of discrete D-flip-flop shift registers. As new pixels stream in via the AXI interface, they are pushed into the bottom right of the shift register grid. From there, they cascade upwards into the first line buffer, flow out of the first line buffer into the middle row of the shift register grid, cascade into the second line buffer, and finally terminate in the top row of the grid. This continuous snake-like progression keeps the necessary pixel data "stationary" on-chip precisely long enough to be reused across three distinct overlapping convolution windows.

Because of this memory architecture, the compute engine can be fully unrolled. It utilizes 9 parallel hardware multipliers connected to a combinatorial adder tree. In a single clock cycle, all 9 pixels in the shift register grid are multiplied by their respective weights, and the products are summed to form the final output pixel. This spatial dataflow perfectly aligns with the mathematical nature of the kernel, minimizing external AXI bus requests to a single byte per cycle while fully saturating the 9 MAC units at a continuous 100 MHz clock rate.

## 5. Hardware Interface

The module communicates with the external system utilizing the industry-standard AMBA AXI4-Stream protocol. We implemented standard AXI4-Stream slave channels ( `s_axis` ) for incoming pixel data and master channels ( `m_axis` ) for the outgoing convolution results.

This interface was selected over alternative protocols, such as AXI4-Full or AXI4-Lite, because convolutions operate on continuous streams of sequential pixel data rather than randomly accessed memory addresses. AXI-Full would require unnecessary overhead for address generation and burst management, while AXI-Lite is too slow for high-throughput datapaths. The AXI4-Stream protocol's `tvalid` and `tready` handshake signals provide robust, cycle-accurate backpressure management. This ensures that the accelerator stalls gracefully and retains its internal state without dropping data if the upstream DMA cannot provide pixels fast enough, or if the downstream consumer becomes temporarily blocked.

At the target 100 MHz clock frequency, the accelerator requires an effective input bandwidth of 100 MB/s to stream 8-bit pixels at a rate of one pixel per cycle. Because the standard AXI bus supports much higher theoretical bandwidths (often exceeding 400 MB/s in modern SoCs), the design is strictly compute-bound by the 100 MHz clock limit of the MAC units, not interface-bound. The interface easily feeds the required 1 byte/cycle to sustain maximum throughput.

## 6. Verification

The RTL implementation was rigorously verified using a self-checking SystemVerilog testbench. Because the hardware interacts directly with an external AXI stream, the verification strategy encompassed both isolated unit testing of the individual dataflow components and full-system co-simulation against known-good M1 software outputs.

The test cases were explicitly designed to stress the edge cases of the architecture. The baseline test verified the initial reset states and the 58-cycle warmup latency required to fill the line buffers before the first `m_axis_tvalid` signal is legally asserted. Subsequent tests covered continuous streaming throughput to ensure the 9 MAC units could sustain one output per cycle without pipeline stalls.

The most critical test cases involved aggressive randomized AXI backpressure. The testbench was configured to randomly toggle the `s_axis_tvalid` and `m_axis_tready` signals. This simulated a heavily congested SoC environment where the DMA drops packets or delays reads. The design successfully held its internal shift register state during these stalls, proving that no pixel data was dropped or overwritten during handshake interruptions. The final M4 simulation successfully processed the full test vector, outputting the expected validation waterfall of `-224` values, perfectly matching the software baseline. *(See Figure 1: Annotated Waveform).*

## 7. Synthesis Results

The verified RTL was synthesized for the Skywater 130nm (Sky130) physical process node using the OpenLane 2 digital implementation flow. The physical synthesis was constrained to a target clock frequency of 100 MHz (a 10.0 ns clock period).

- **Timing:** The design successfully closed timing with a positive Setup and Hold worst-case negative slack (WNS). As anticipated during the architectural design phase, the critical path lies entirely within the combinatorial logic of the compute core. Specifically, the path begins at the D-flip-flops of the 3x3 shift register grid, travels through the 8-bit multipliers, and cascades

through the deeply nested 9-input adder tree before terminating at the final 32-bit output accumulation register.

- **Area:** The total standard cell area is **32070.8 um^2**, extracted directly from the OpenLane `metrics.csv`. While the MAC units are dense, the dominant area contributors are the synthetic SRAM/DFF arrays required for the two internal line buffers. Storing two full rows of image data requires significantly more physical silicon real estate than the combinational math logic.
- **Power:** The estimated total power is **6.27 mW (6.273142e-03 Watts)**, derived from the post-routing OpenLane power report. The power consumption is heavily dominated by dynamic switching power. Because the architecture relies on a continuous sliding window, data in the shift registers and line buffers is physically moving every single active clock cycle, causing high toggle rates across the entire memory hierarchy and the massive internal clock tree network required to synchronize it.

## 8. Benchmark Results

The hardware implementation drastically outperformed the initial software estimation, validating the decision to move the kernel to custom silicon.

- **Baseline:** The M1 Software baseline achieved ~0.05 GOPS at an estimated energy cost of ~45.0 mJ per frame.
- **Accelerator:** The M4 Hardware accelerator achieved exactly 1.8 GOPS at an estimated energy cost of ~2.1 mJ per frame.

This represents an immense **36x speedup** over the software baseline. Notably, the gap between the measured simulation performance and the theoretical peak performance is virtually zero during steady-state operation. Once the 58-cycle line buffer warmup penalty is paid at the start of a frame, the accelerator produces exactly one valid convolution result per clock cycle indefinitely.

The energy reduction is a direct mathematical consequence of executing the workload 36 times faster. While the instantaneous power draw of the custom silicon might be higher than the CPU when fully active, the workload is completed in a fraction of the time. This heavily reduces the duration the system spends leaking static power and dramatically lowers the total Joules required per image frame.

## 9. What Did Not Work

During the transition from the M2 algorithmic model to the fully pipelined M3/M4 RTL architecture, several significant design challenges arose, primarily surrounding AXI interface synchronization and internal state management.

Our initial implementation of the shift-register grid failed to properly account for downstream AXI backpressure. The dataflow was designed to blindly shift pixels horizontally and vertically every clock cycle. When we injected simulated backpressure by deasserting the `m_axis_tready` signal in the testbench (simulating a busy receiver), the internal shift registers kept moving data. This caused the oldest pixels in the 3x3 window to be overwritten and permanently lost before the stalled output could be accepted. We learned that a pipeline cannot be locally controlled; the clock enable signals for the entire memory hierarchy, including the line buffers and the shift register grid, must be strictly gated by the boolean condition `(s_axis_tvalid && m_axis_tready)`.

Additionally, we struggled with the boundary conditions during the initial line buffer fill. We initially attempted to output data while the buffers were only partially full, resulting in corrupted garbage data for the first two rows of the image. We had to implement a dedicated latency counter that explicitly disables `m_axis_tvalid` until exactly 58 valid input pixels have been consumed.

If we were to redesign this architecture in the future, we would implement a true dual-clock asynchronous FIFO interface at the AXI boundaries rather than relying purely on synchronous clock gating. While our synchronous gating works flawlessly for this specific 100 MHz target, an asynchronous FIFO would decouple the compute core's timing domain from the SoC's bus frequency, making the IP block significantly more modular and easier to integrate into varied physical systems.

---

## Appendix: Figures

**Figure 1:** Final annotated simulation waveform showing AXI handshakes and valid data output. (Reference: `project/m4/report/figures/final_waveform.png` ) **Figure 2:** Final Roofline plot demonstrating the shift from memory-bound to compute-bound performance. (Reference: `project/m4/report/figures/roofline_final.png` )